Hotel Reservation DBMS

Bookings, room changes and stay continuations

Group 60: Jinxing Wu, Tianyi Shen, Zirui Meng

THE UNIVERSITY OF SYDNEY

COMP9120 · Semester 2 2026

GitHub repository
github.com/jinxwu2026-ship-it/
COMP9120_Hotel_Reservation_System

7 Relations
48 attributes total

7 Foreign keys
Core, staff and audit links

54 Application tests
17 + 14 + 18 + 5 checks

200 Seed reservations
Across 40 seeded rooms

Room changes, reservations and audit history commit together—or roll back together.

01 Why this project

Busy holiday periods sparked our curiosity about hotel booking challenges.

Built for hotel receptionists and managers.

02 Business workflows

Atomic multi-room booking

All selected room bookings succeed together or roll back together.

Manager-confirmed discounts

Fixed agreed rates need manager approval and a recorded reason.

Attributed history

Reservation changes retain staff identity and before/after snapshots.

03 Integrity rules

Contacts ≠ occupants

Guest is the booking contact. ReservationGuest lists the people staying.

Two uniqueness guards

One checked-in stay per room.
One non-cancelled successor per stay.

Active-date overlap

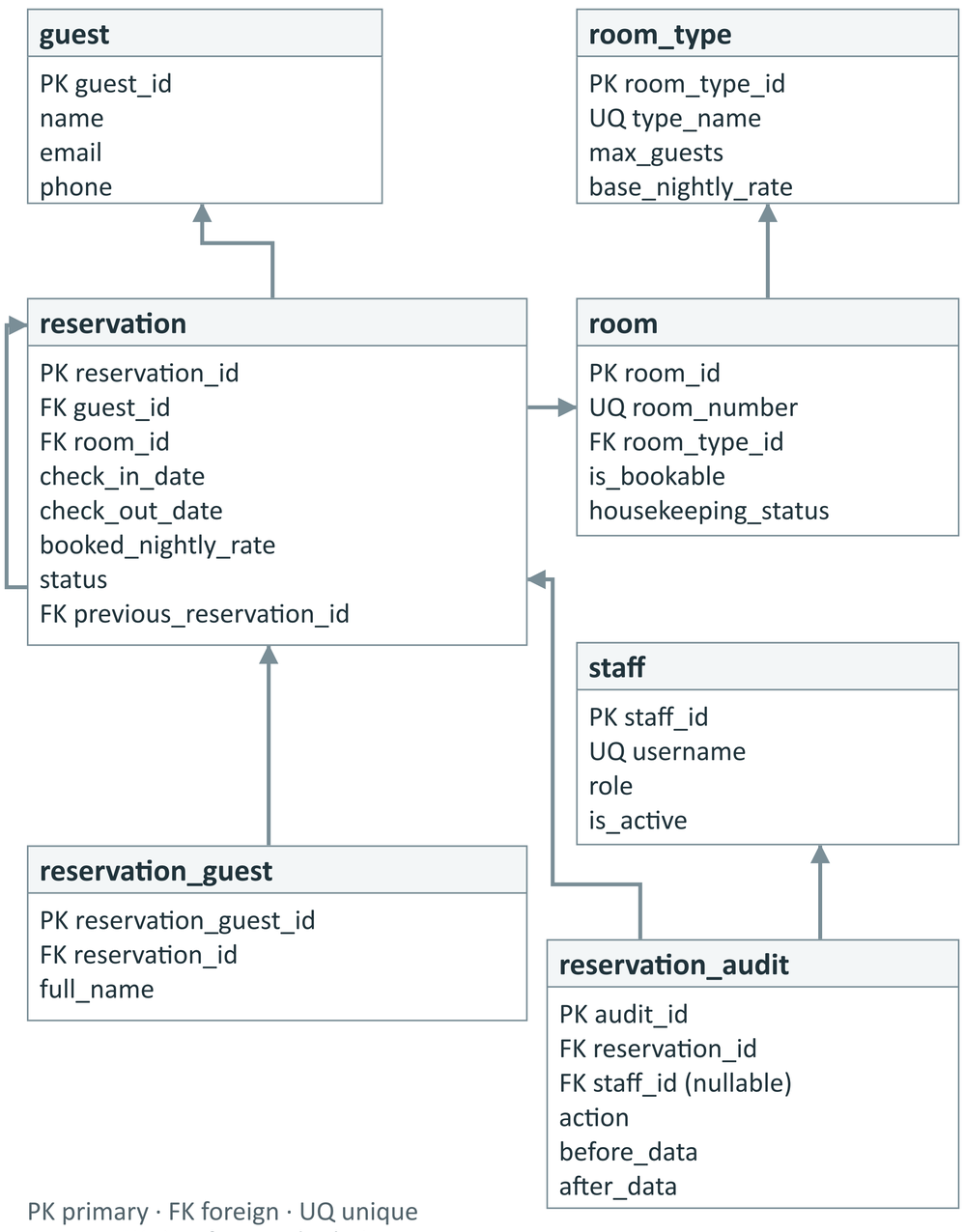
```
r.status IN ('confirmed',
'checked_in')
AND r.check_in_date < %s
AND r.check_out_date > %s
```

Parameters: requested checkout, then requested check-in.

Date indexes aid lookup; room locks and rechecks prevent conflicts.

04 Relational model overview

Key fields shown; complete RM in the report.



PK primary · FK foreign · UQ unique
Arrows: FK → referenced relation.

05 Atomic continuation

Create the next confirmed segment first.
Activation is a separate transaction.

01 Lock rooms and stays in ID order
▼
02 End the previous stay
▼
03 Mark old room dirty if changing rooms
▼
04 Start next stay at the same timestamp
▼
05 Write both reservation audit events
▼
06 **COMMIT**

Any failed step

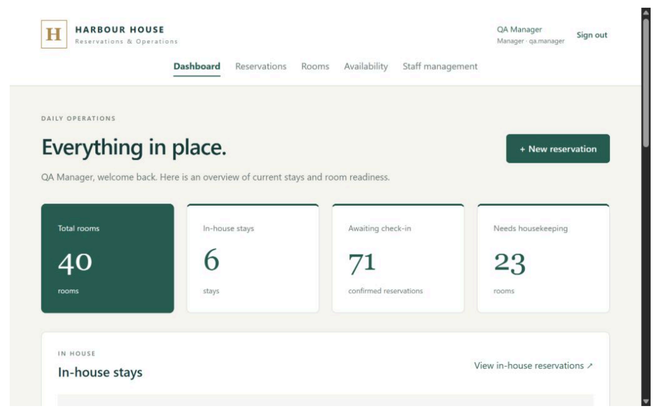
ROLLBACK all changes

Old checkout = new check-in timestamp.

Previous: checked_in → checked_out
Next: confirmed → checked_in

Same-room continuation keeps the room clean.

06 Working prototype



Web + CLI → Python services → PostgreSQL

An English portal for receptionists and managers.

07 Concurrent requests

Two sessions request the same room and overlapping dates.

Time	Session A	Session B
1	Lock room	Wait
2	Check dates	Wait
3	Insert; COMMIT	Acquire lock
4	—	Recheck; reject

READ COMMITTED

One accepted, one refused.
Recheck availability after waiting.

08 Verified behaviour

Test category	Passed
Application tests	54
Constraint / positive	43
Standalone SQL cases	35

Real PostgreSQL, rollback and competing connections.

09 Takeaways

Linked stay history.
Atomic room handover.
Controlled concurrent writes.

Acknowledgements

AI assisted SQL, setup, synthetic data, the demonstration web interface, report preparation and poster design. The group remains responsible for the work.

Sources: Group 60 report, implemented schema and test evidence.